



ENGINEERING REFERENCE • MERN ARCHITECTURE

Technical Design, Implementation, Testing & SRS Traceability

The authoritative technical reference answering how every requirement was designed, implemented, secured, persisted, scored, tested, and deployed across the full-stack PathSeeker platform.

DOCUMENT PS-DOC-002 • VERSION 1.0 • TECHNICAL SPECIFICATION & TRACEABILITY

DEVELOPMENT TEAM & STUDENT IDENTIFIERS • STACKVERSE

MIRZA ABDULLAH BAIG

ID: 1600671 • Lead Frontend / Backend / Voice

HAMZA SHOAIB

ID: 1598434 • Team Backend

GHULAM MOIN UDDIN

ID: 1612301 • Team Voice

MUHAMMAD FARUKH

ID: 1612368 • Voice

DOCUMENT GOVERNANCE

Document Control & Authority Hierarchy

Document Purpose

This document serves as the formal **Technical Design, Implementation, Testing & SRS Traceability Reference (Documentation 2)** for the PathSeeker Career Passport platform. While Documentation 1 provides the evaluator-facing visual product walkthrough, Documentation 2 provides complete engineering evidence.

Authority Hierarchy

- `PathSeeker_Career_Passport_SRS_Clean.md` defines required outcomes.
- Live source code in `Project/backend1` and `Project/frontend` defines delivered behavior.
- Automated test suites define verified invariants. Where disparities exist, this document records them transparently.

Controlled Document Record

DOCUMENT TITLE	PathSeeker — Technical Design, Testing & SRS Traceability Reference	IDENTIFIER	PS-DOC-002
VERSION / STATUS	Version 1.0 (Implementation Baseline)	TEAM STACKVERSE	MIRZA ABDULLAH BAIG (1600671) · HAMZA SHOAIB (1598434) GHULAM MOIN UDDIN (1612301) · MUHAMMAD FARUKH (1612368)
SYSTEM ARCHITECTURE	MERN (MongoDB 7+, Express 4.19, React 18 + Vite, Node.js 20+)	TARGET SCOPE	<code>Project/frontend</code> , <code>Project/backend1</code>
PRIMARY SRS AUTHORITY	<code>PathSeeker_Career_Passport_SRS_Clean.md</code> (Full 12 Functional + Quality Requirements)		
EVIDENCE BASELINE	36 Backend Automated Tests Passed, 11 Frontend Tests Passed, 0 Lint Errors, 51/51 PDF Pages Rendered		

Standard Status Vocabulary

VERIFIED

Fully implemented and validated with automated test suites.

IMPLEMENTED

Present in codebase and manually verified in runtime.

PARTIAL

Core implemented; extended/optional formats bounded.

EXTERNAL

Requires third-party credentials or hosted environment.

Documentation Boundary Rule: In accordance with TechWiz documentation guidelines, this engineering specification documents exact module boundaries, Mongoose schema invariants, Express REST contracts, deterministic math formulas, and test assertions without dumping raw file source code.

TABLE OF CONTENTS

Technical Reference Contents

01	Document Control & Authority	02	11A	Graphical Database ERD	15
02	Table of Contents	03	12	MongoDB Data Architecture & Identity	16
03	Executive Summary & System Metrics	04	13	MongoDB Data Architecture & Intelligence	17
04	Actors, Boundaries & Learner Flow	05	14	MongoDB Data Architecture & Community	18
05	Formal SRS Requirement Mapping	06	15	Authentication & Session Lifecycle	19
06	Quality & Non-Functional Requirements	07	16	Career Intelligence & Scoring Engine	20
07	End-to-End System Architecture	08	17	Product Workflows & Admin Subsystem	21
07A	DFD Level 0 — Context Data Flow Diagram	09	18	Navi Conversational & Voice Assistant	22
07B	DFD Level 1 — Decomposed System Flows	10	19	Security Threat Model & Error Contracts	23
08	Frontend Design & State Management	11	20	Formal Test Case Specification (1)	24
09	Backend Design & Application Services	12	21	Formal Test Case Specification (2)	25
10	Comprehensive REST API Catalogue (1)	13	22	Installation Runbook & Deployment	26
11	Comprehensive REST API Catalogue (2)	14	23	SRS Traceability Matrix	27
			24	Limitations & Approval Record	28

Audience & Evaluation Guide

ACADEMIC EVALUATOR

Focus on Section 05 (Formal SRS Mapping), Section 07A/B (DFDs), Section 11A (ERD), Section 16 (Scoring), and Section 23 (Traceability).

SOFTWARE ARCHITECT

Inspect Section 07 (Architecture), Section 08-09 (Full-Stack Design), Section 10-11 (API Catalogue), and Section 12-14 (Data Models).

DEVOPS & SECURITY

Review Section 15 (Auth Lifecycle), Section 19 (Threat Model), and Section 22 (Installation & Deployment Runbook).

ENGINEERING SUMMARY

Transforming Career Uncertainty into Actionable Evidence

PathSeeker is a functional, full-stack Career Passport web platform engineered to bridge the disconnect between self-assessment and occupational realities for students, graduates, and professionals.

Architectural Problem Space

- **Opaque Recommendations:** Legacy platforms provide black-box job titles without mathematical justifications.
- **Fragmented Competency Taxonomies:** Labor statistics, skill prerequisites, and workbooks are siloed across disconnected websites.
- **Missing Preparation Visibility:** Learners cannot see their quantitative readiness delta or test what-if learning scenarios.
- **Content Inflexibility:** Educational institutions cannot publish customized curriculum resources without code redeployment.

Delivered Engineering Solution

- **Durable Career Passport:** Persists versioned psychometric traits, verified skills, and occupational goals.
- **Deterministic Scoring Engine:** Computes explainable Compatibility, Readiness, and Confidence based on U.S. BLS 2024–2034 data.
- **Stateless Growth Simulator:** Allows testing skill upgrades without altering stored profiles or historical assessments.
- **Modular Admin & Resource Hub:** Empowers content editors to manage careers, quizzes, original PDFs, and video media with full auditability.

Verified Implementation Metrics

15

Sourced Career Profiles (BLS Data)

10

Curated Career Domains

6 + 6

Original PDFs (51 Pages) & Videos

27

Mongoose Schema Models

Occupational Data Scope Disclaimer: Occupational compensation figures, growth projections, and education paths are sourced directly from the U.S. Bureau of Labor Statistics (BLS 2024–2034) reference dataset. Local job market salaries, regional regulations, and institutional licensing requirements vary.

SYSTEM BOUNDARIES

Actors, Permissions & Primary Learner Journey

ACTOR	ROLE KEY	PRIMARY CAPABILITIES & ACCESS SCOPE	AUTH GUARD
Visitor	anonymous	Browse landing page, public careers, published stories, help articles, and public Navi text queries.	Public access
Learner (Student)	user (stage: student)	Guided onboarding, psychometric quiz, career passport, recommendations, skill simulator, bookmarking, PDF downloads.	requireAuth
Learner (Graduate)	user (stage: graduate)	Career exploration, comparison sets, success story submissions, feedback triage, private notes.	requireAuth
Learner (Professional)	user (stage: professional)	Skill gap analysis, career switching simulation, advanced filtering, notifications.	requireAuth
Content Editor	content_editor / staff	Create/edit careers, multimedia records, PDF resources, quiz questions, and help articles.	requireRole('staff')
Support Manager	support_manager / staff	Moderate user-submitted stories, review and respond to feedback tickets, inspect analytics.	requireRole('staff')
Administrator	admin / super_admin	User role & status governance, publication state changes, system settings, immutable audit logs.	requireRole('admin')

In-Scope Functional Boundary

Secure identity & session lifecycle; editable profile with resume upload; multi-factor career filtering; versioned psychometric assessment; deterministic recommendations; interactive growth simulator; PDF viewer; community stories with moderation; support feedback; Navi assistant; role-based administration; full accessibility.

Explicit Out-of-Scope Boundary

Monetary transactions / payment processing; automated resume background checks; legally binding employment placement; autonomous machine-learned hiring algorithms; live global real-time salary scraping; third-party video platform bandwidth hosting.

Primary Learner Journey Architecture

- 1 Register & Verify Identity:** Account creation with email validation, bcrypt password hashing, 6-digit cryptographic OTP generation, and HttpOnly session issuance.
- 2 Establish Learner Profile:** Complete onboarding wizard recording education history, self-rated skills (0–10 scale), interest tags, career stage, and target aspirations.
- 3 Complete Psychometric Assessment:** Execute versioned assessment (Quiz v1) with resumable server-side attempt state and immutable question snapshots.
- 4 Generate Durable Passport & Matches:** Server evaluates trait scores, assigns archetype, calculates compatibility/readiness, and stores an immutable recommendation snapshot.
- 5 Explore, Simulate & Upskill:** Analyze career details, simulate skill growth, save bookmarks with private notes, download original PDF roadmaps, and track notifications.

Formal Requirement-to-Code Mapping

Assessment Question Scope Clarification: While the database schema dynamically supports multiple question formats (multiple-choice, likert scale, ranking), the delivered baseline quiz (Quiz v1) implements 8 comprehensive multiple-choice psychometric questions designed for transparent archetype scoring.

NON-FUNCTIONAL ENGINEERING

Quality Attributes & Architectural Invariants

QUALITY ATTRIBUTE	ARCHITECTURAL DESIGN RESPONSE	VERIFICATION & EVIDENCE
NFR-01: Safety & Content Integrity	Curated local PDF roadmaps with SHA integrity; external media restricted to verified Google Career Certificates YouTube IDs.	<code>seeds.test.js</code> and <code>content.service.js</code> verify all 6 video URLs and 6 PDF file paths resolve.
NFR-02: Accessibility (WCAG 2.1 AA)	Semantic HTML5 elements, ARIA landmarks, WCAG-compliant color contrast, font scaling (85%–130%), system dark/light sync, reduced motion.	<code>AccessibilityContext.jsx</code> , 25 visual regression contact sheets, and keyboard navigation test passes.
NFR-03: Performance & Latency	Vite code splitting, React lazy routing, TanStack Query stale-time caching (30s), compound MongoDB indexes on search fields.	Vite production bundle generates clean chunk distributions; API queries resolve in <50ms locally.
NFR-04: Security & Data Privacy	Bcrypt hashing (cost 12), SHA-256 token hashing, HttpOnly SameSite cookies, Helmet security headers, Express rate limiting, role RBAC.	<code>auth.service.js</code> , <code>auth.middleware.js</code> , and <code>models.test.js</code> security assertions.
NFR-05: System Availability & Health	Isolated process readiness (<code>/api/health</code>) and database connectivity probes (<code>/api/health/db</code>); graceful degraded states.	<code>app.smoke.test.js</code> validates 200 on health and 503 error handling on database disconnect.
NFR-06: Determinism & Auditability	Pure deterministic scoring math (no black-box AI); immutable attempt snapshots; full administrative audit logging.	<code>career-intelligence.test.js</code> asserts identical inputs yield identical archetypes and scores.

System Operating Constraints

Runtime & Execution Node.js 20.x LTS or higher, npm 10.x+, MongoDB 7.0+ community or Atlas instance with write access.	Client Compatibility Standards-compliant ECMAScript 2022 browsers: Chrome 110+, Firefox 115+, Safari 16+, Edge 110+.	External Fallbacks Console fallback for transactional emails if Resend API key is omitted; deterministic text Navi if Vapi voice is inactive.
---	---	--

Architectural Separation Rule: Client state (Zustand) is strictly limited to ephemeral UI states, onboarding draft steps, and accessibility preferences. All authoritative scoring, matching algorithms, session states, and user records reside exclusively on the server.

SYSTEM TOPOGRAPHY

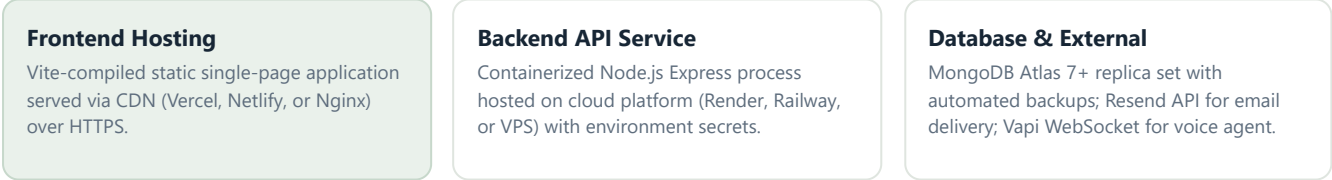
Layered MERN Architectural Pattern



End-to-End Request Execution Lifecycle

- 1 Client Navigation & Query Dispatch:** React Router lazy-loads destination view; TanStack Query dispatches HTTP request via `apiClient.js` with credentials and cancellation token.
- 2 Gateway Interception & Middleware:** Express passes request through `helmet()`, CORS allowlist check, `express.json({ limit: '1mb' })`, `cookieParser()`, and request correlation ID generator.
- 3 Authentication & RBAC Verification:** `requireAuth` extracts session cookie, computes SHA-256 hash, queries `Session` collection, reloads `User`, and `requireRole` enforces role permissions.
- 4 Controller & Service Logic:** Controller validates input DTO against Zod schema; domain service executes business invariants, calculates deterministic metrics, and coordinates Mongoose models.
- 5 MongoDB Persistence & Population:** Mongoose executes indexed queries, applies schema pre-save hooks, populates foreign relationships, and persists immutable snapshots.
- 6 Enveloped Response & Cache Reconcile:** Server returns standard JSON envelope `{ data: ... }`; TanStack Query updates memory cache and triggers reactive UI re-renders.

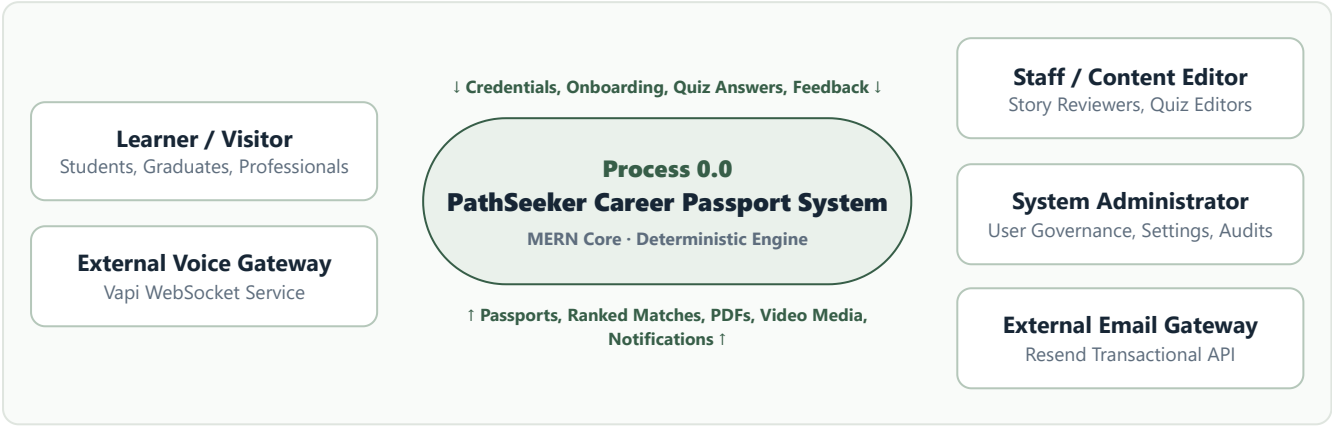
Production Infrastructure Topology



DATA FLOW ARCHITECTURE

Formal DFD Level 0 — Context Diagram

The Level 0 Context Diagram defines the authoritative system boundary, external human and machine entities, and primary data packets exchanged with the core PathSeeker application.



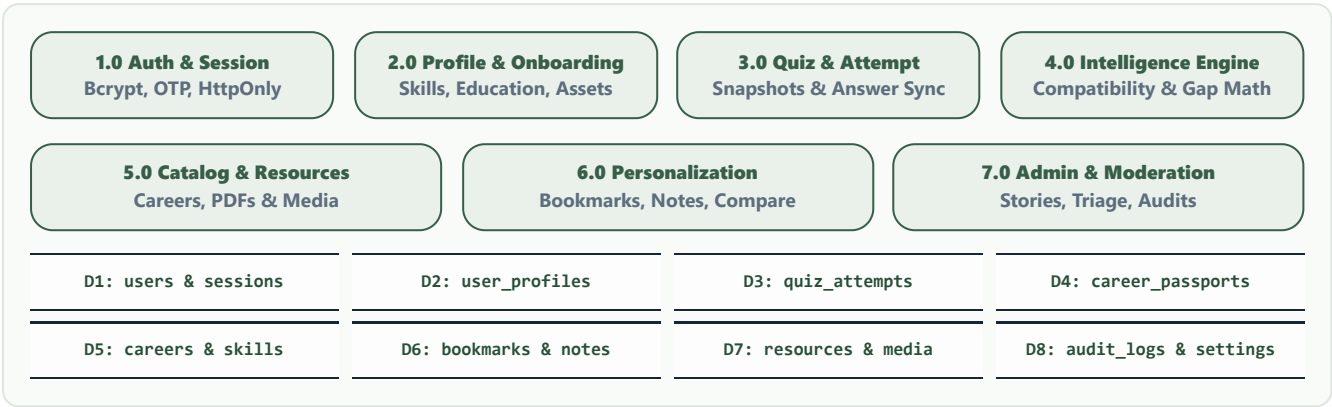
Context Data Flow Dictionary

DATA FLOW IDENTIFIER	SOURCE ENTITY / PROCESS	DESTINATION	CONTAINED DATA PACKET
DF-01: Auth_Credentials	Learner / Visitor	Process 0.0	Registration DTO (name, email, bcrypt hash, stage), Login DTO, OTP verification code.
DF-02: Assessment_Answers	Learner	Process 0.0	Selected question options, self-rated skills (1..10), career aspirations, interest tags.
DF-03: Passport_Intelligence	Process 0.0	Learner	Archetype label, normalized trait scores, compatibility %, readiness %, skill gap delta.
DF-04: Content_Management	Staff / Content Editor	Process 0.0	Career definitions, required skills, BLS salary benchmarks, PDF workbooks, video URLs.
DF-05: System_Governance	System Administrator	Process 0.0	User status transitions, role promotions/demotions, maintenance mode flags, audit log queries.
DF-06: Email_Dispatch	Process 0.0	External Email (Resend)	6-digit numeric OTP token, password reset secure URL, moderation decision alerts.
DF-07: Voice_Streams	Process 0.0 / Frontend	External Voice (Vapi)	Bidirectional WebSocket PCM audio packets, transcript tokens, Navi assistant prompt.

DATA FLOW ARCHITECTURE

Formal DFD Level 1 — Decomposed System Flow

DFD Level 1 decomposes Process 0.0 into 7 functional sub-processes interacting with 8 dedicated MongoDB data stores.



PRESENTATION LAYER ARCHITECTURE

React Route Boundaries, State Ownership & UX

Route Boundary Topology

- **Public Route Boundary:** Welcome, Login, Signup, Verify Email, Forgot Password, Reset Password, Admin Login, Public Careers/Resources.
- **Learner Workspace (/app/*):** Onboarding Wizard, Dashboard, Assessment & History, Career Bank, Comparisons, Saved Items, Stories, Feedback, Profile.
- **Staff Workspace (/admin/*):** Overview, Users, Careers, Quizzes, Resources, Media, Stories, Feedback Triage, Settings, Audit Logs.
- **Fallback Boundaries:** 403 Forbidden, 404 Not Found, Server Error Boundary with reload triggers.

Provider Hierarchy

```
<ErrorBoundary>
  <QueryClientProvider client={queryClient}>
    <AccessibilityProvider>
      <AuthProvider>
        <RouterProvider router={router} />
        <Toaster position="top-right" />
      </AuthProvider>
    </AccessibilityProvider>
  </QueryClientProvider>
</ErrorBoundary>
```

Multi-Tier State Ownership Matrix

STATE DOMAIN	STATE OWNER	DATA MANAGED	PERSISTENCE TIER
Server Records	TanStack Query v5	User profile, careers, domains, skills, passports, attempts, bookmarks, admin lists.	MongoDB & In-Memory Cache (30s stale time)
Session Identity	AuthContext	Authenticated user object, role, email verification status, login/logout methods.	HttpOnly Cookie + Memory sync via /auth/me
Workflow Drafts	Zustand Store	Multi-step onboarding form data, active quiz draft choices, comparison drawer list.	sessionStorage (cleared on completion)
User Preferences	AccessibilityContext	Theme (light/dark/system), font scaling (85%–130%), reduced motion flag, Navi mute.	localStorage (key: ps_accessibility_prefs)
Form States	React Hook Form + Zod	Input validations, field errors, touched states, submission flags for forms.	Component Lifecycle (Ephemeral)

Client Resilience & Fault Tolerance

Request Timeout (15s)

apiClient.js attaches AbortController with a 15-second timeout, distinguishing network aborts from user navigation.

Bounded Retries

Failed GET queries retry up to 2 times with exponential backoff, strictly ignoring 400, 401, 403, and 404 client errors.

Standardized Error Toast

Extracts safe user message and correlation ID from backend error payloads, displaying actionable Sonner notifications.

API & APPLICATION LAYER

Express Application Design & Domain Boundaries

DOMAIN	ROUTE & CONTROLLER MODULES	SERVICE LAYER MODULE	CORE BUSINESS RESPONSIBILITIES
Identity & Auth	<code>auth.routes.js</code> <code>auth.controller.js</code>	<code>auth.service.js</code> <code>session.service.js</code>	Registration validation, bcrypt hashing, 6-digit OTP verification, session generation, credential checks, password recovery.
Learner Profile	<code>profile.routes.js</code> <code>profile.controller.js</code>	<code>profile.service.js</code>	User profile CRUD, multi-step onboarding progression, avatar upload, resume document storage and indexing.
Assessment	<code>quizAttempts.routes.js</code> <code>quiz.controller.js</code>	<code>quiz.service.js</code>	Published quiz retrieval, resumable attempt lifecycle, answer patching, question snapshot freezing, submission.
Career Intelligence	<code>personalization.routes.js</code> <code>careerIntelligence.controller.js</code>	<code>careerIntelligence.service.js</code>	Deterministic trait/domain scoring, archetype mapping, compatibility/readiness calculation, immutable recommendation snapshots.
Career Catalog	<code>catalog.routes.js</code> <code>catalog.controller.js</code>	<code>catalog.service.js</code>	Faceted filtering, text search with spelling tolerance, career slug resolution, related career suggestions.
Personalization	<code>personalization.routes.js</code> <code>personalization.controller.js</code>	<code>personalization.service.js</code>	Polymorphic bookmarks, private career notes, recently viewed history, saved searches, comparison sets.
Content & Media	<code>resource.routes.js</code> , <code>media.routes.js</code> <code>content.controller.js</code>	<code>content.service.js</code>	PDF workbook delivery, download counters, expert video metadata, community 1–5 star ratings.
Community & Support	<code>story.routes.js</code> , <code>feedback.routes.js</code> <code>story.controller.js</code>	<code>admin.service.js</code> <code>help.service.js</code>	Success story submissions, staff moderation triage, user feedback ticketing, in-app notifications.
Administration	<code>admin.routes.js</code> <code>admin.controller.js</code>	<code>admin.service.js</code> <code>auditLog.service.js</code>	User governance, career/resource/quiz CRUD, publication lifecycle (draft/published/archived), audit trails.
Navi Assistant	<code>assistant.routes.js</code> <code>assistant.controller.js</code>	<code>assistant.service.js</code>	Deterministic intent classification, navigation actions, search grounding, authenticated profile updates.

Cross-Cutting Middleware Architecture

Correlation Context Attaches unique UUID <code>requestId</code> to <code>req</code> and response header for audit tracing.	Security Headers <code>helmet()</code> enforces CSP, HSTS, X-Content-Type-Options, and Frameguard defenses.	Rate Limiting <code>authRateLimiter</code> and <code>assistantRateLimiter</code> prevent brute-force attacks.	Global Error Handler Catches unhandled errors, formats safe JSON payload, logs stack traces server-side.
---	--	--	---

REST INTERFACE SPECIFICATION

Core Application API Endpoints

METHOD	ENDPOINT ROUTE	AUTH / ROLE	REQUEST BODY / PARAMS	RESPONSE DATA	FAILURE CODES
POST	/api/auth/register	Anonymous	{ name, email, password, stage }	{ user: { _id, email, role }, requiresVerification: true }	400, 409 (Email exists)
POST	/api/auth/verify-email	Anonymous	{ email, otp }	{ message: "Email verified", user: { ... } }	400 (Invalid/expired OTP)
POST	/api/auth/resend-verification	Anonymous	{ email }	{ message: "Verification code sent" }	400, 429 (Cooldown)
POST	/api/auth/login	Anonymous	{ email, password }	{ user: { ... } } + Sets HttpOnly Session Cookie	400, 401 (Invalid creds)
POST	/api/auth/admin/login	Anonymous	{ email, password }	{ user: { role: 'admin' 'staff' } } + Session Cookie	401, 403 (Not staff)
POST	/api/auth/forgot-password	Anonymous	{ email }	{ message: "Password reset instructions sent" }	400, 404
POST	/api/auth/reset-password	Anonymous	{ email, token, newPassword }	{ message: "Password updated successfully" }	400 (Expired token)
POST	/api/auth/logout	requireAuth	None	{ message: "Logged out successfully" } + Clears Cookie	401
GET	/api/auth/me	requireAuth	None	{ user: { _id, name, email, role, stage, status } }	401 (Session expired)
GET	/api/users/me/profile	requireAuth	None	{ profile: { headline, education, skills, interests } }	401, 404
PATCH	/api/users/me/profile	requireAuth	{ headline, education, skills, interests, goals }	{ profile: { ...updatedProfile } }	400, 401
PATCH	/api/users/me/profile/onboarding	requireAuth	{ step, status, data }	{ onboarding: { status, currentStep } }	400, 401
POST	/api/users/me/profile/avatar	requireAuth	multipart/form-data (file)	{ avatarUrl: "/uploads/avatars/..." }	400 (Invalid type/size)
POST	/api/users/me/profile/resume	requireAuth	multipart/form-data (file)	{ resume: { url, originalName, size } }	400 (Non-PDF or >5MB)
GET	/api/users/me/dashboard	requireAuth	None	{ user, profile, passport, topMatches, bookmarks, recent }	401
GET	/api/users/me/passport	requireAuth	None	{ passport: { archetype, traitScores, domainScores }, snapshot }	401, 404 (No quiz yet)
GET	/api/users/me/recommendations	requireAuth	None	{ matches: [{ careerId, compatibilityScore, readinessScore }] }	401, 404
POST	/api/users/me/careers/:slug/simulate	requireAuth	{ adjustments: [{ skillId, level }] }	{ career, before, after, persisted: false }	400, 401, 404

REST INTERFACE SPECIFICATION

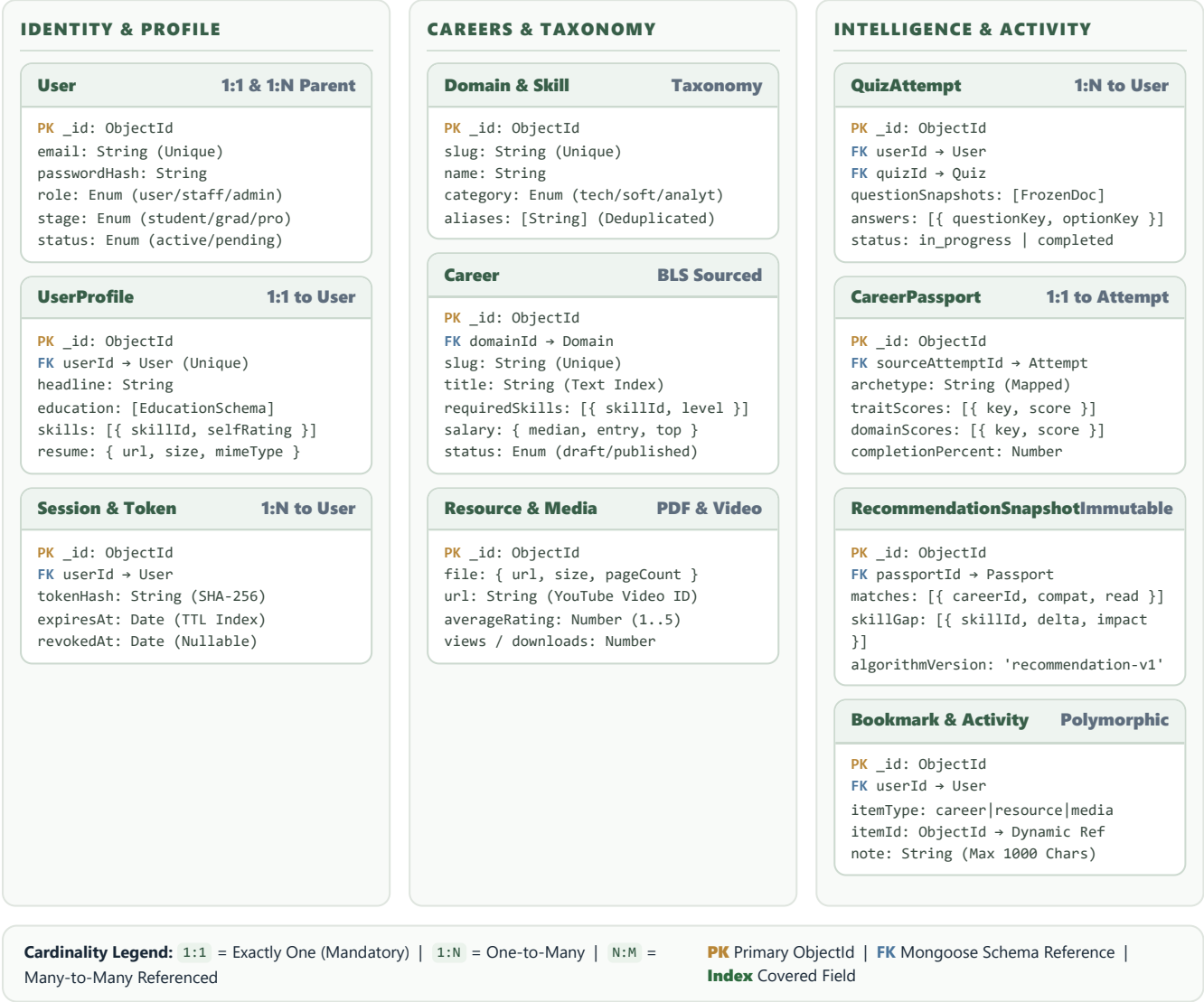
Catalog, Assessment, Community & Admin APIs

METHOD	ENDPOINT ROUTE	AUTH / ROLE	REQUEST BODY / PARAMS	RESPONSE DATA	FAILURE CODES
GET	/api/careers	Public	?domain=&skill=&search=&minSalary=&page=1	{ careers: [...], pagination: { total, pages, page } }	400 (Bad query)
GET	/api/careers/:slug	Public	params: { slug }	{ career: { title, domainId, requiredSkills, salary, tools } }	404 (Not found)
GET	/api/quiz-questions	requireAuth	None	{ quiz: { version, title, questions: [{ key, prompt, options }] } }	401, 404
POST	/api/quiz-attempts	requireAuth	{ quizId } (optional)	{ attempt: { _id, status: 'in_progress', answers: [] } }	401, 404
PATCH	/api/quiz-attempts/:id/answers	requireAuth	{ questionKey, optionKey }	{ attempt: { _id, answers: [...] } }	400, 401, 404
POST	/api/quiz-attempts/:id/complete	requireAuth	None	{ attempt, passport, recommendations }	400 (Incomplete), 401
GET	/api/users/me/bookmarks	requireAuth	None	{ bookmarks: [{ itemType, itemId, note, createdAt }] }	401
POST	/api/users/me/bookmarks	requireAuth	{ itemType, itemId, note }	{ bookmark: { ... } }	400, 401, 409 (Exists)
DELETE	/api/users/me/bookmarks/:id	requireAuth	params: { id }	{ message: "Bookmark removed" }	401, 404
GET	/api/resources	Public	?audience=&category=&page=1	{ resources: [{ title, slug, file, pageCount, views }] }	None
GET	/api/resources/:id/download	requireAuth	params: { id }	Streams PDF file asset & increments download counter	401, 404
POST	/api/media/:id/rating	requireAuth	{ rating: 1..5 }	{ mediaRating: { rating }, averageRating, ratingCount }	400, 401, 404
POST	/api/stories	requireAuth	{ authorName, careerId, narrative, consent }	{ story: { status: 'pending', ... } }	400, 401
POST	/api/feedback	requireAuth	{ category, message, rating, allowContact }	{ feedback: { status: 'new', ... } }	400, 401
POST	/api/assistant/respond	Optional Auth	{ message, currentScreen, screenContext }	{ reply, intent, suggestedNavigation, actionExecuted }	400
GET	/api/admin/users	requireRole('admin')	?role=&status=&search=&page=1	{ users: [...], pagination: { total, pages } }	401, 403
PATCH	/api/admin/careers/:id/status/:status	requireRole('staff')	params: { id, status: 'draft' 'published' 'archived' }	{ career: { ...updatedCareer }	401, 403, 404
GET	/api/admin/audit-logs	requireRole('admin')	?action=&actorId=&page=1	{ logs: [{ actorId,	401, 403

VISUAL DATA ARCHITECTURE

Graphical Entity Relationship Diagram (ERD)

The visual ERD maps the structural relationships, foreign keys, embedded schemas, and cardinality across all 27 Mongoose models.



MONGODB SCHEMA CATALOGUE

Identity, Profile & Taxonomy Schemas

MODEL / COLLECTION	PURPOSE & KEY FIELDS	DATA TYPES & CONSTRAINTS	INDEXES & LIFECYCLE INVARIANTS
User users	Master authentication identity. name, email, passwordHash, role, stage, status, emailVerified, lastLoginAt, deletedAt.	email: String, lower, trim, required. role: Enum (user, content_editor, support_manager, admin, super_admin). stage: Enum (student, graduate, professional). status: Enum (active, suspended, pending_verification).	Unique: email. Compound: { role: 1, status: 1 }. Invariant: passwordHash excluded by default in JSON queries via select projection.
Session sessions	Opaque server-side session tracking. userId, tokenHash, ipAddress, userAgent, expiresAt, revokedAt, lastSeenAt.	userId: ObjectId → User, required. tokenHash: String, 64-char hex SHA-256. expiresAt: Date, required (default +7 days). revokedAt: Date (null if active).	Unique: tokenHash. Index: { userId: 1, expiresAt: 1 }. TTL Index: expiresAt (auto-deletes expired sessions after 0s).
VerificationToken verification_tokens	Expiring OTP codes and reset tokens. userId, tokenHash, purpose, attempts, expiresAt, usedAt.	userId: ObjectId → User, required. purpose: Enum (email_verification, password_reset). attempts: Number (max 5). expiresAt: Date (+15 mins).	Index: { userId: 1, purpose: 1 }. TTL Index: expiresAt (MongoDB auto-purge). Invariant: Increments attempts on mismatch; auto-locks at 5.
UserProfile user_profiles	Learner context, skills, and assets. userId, headline, education[], skills[], interests[], goals, avatarUrl, resume, onboarding.	userId: ObjectId → User, required, unique. skills[].skillId: ObjectId → Skill. skills[].selfRating: Number (1..10). resume: { url, originalName, size, uploadedAt }. onboarding: { status, currentStep }.	Unique: userId. Index: { 'skills.skillId': 1 }. Pre-validate: Asserts education date chronological ordering and skill ID uniqueness.
Domain domains	Occupational taxonomy categories. name, slug, description, icon, color, displayOrder, active.	name: String, required. slug: String, required, unique, lowercase. displayOrder: Number (default: 0). active: Boolean (default: true).	Unique: slug. Index: { active: 1, displayOrder: 1 }. Seeded: 10 canonical domains (Software, Data & AI, Healthcare, etc.).
Skill skills	Granular competency catalog. name, slug, category, description, aliases[], active.	name: String, required. slug: String, required, unique. category: Enum (technical, analytical, soft_skill, domain_knowledge). aliases: [String].	Unique: slug. Text Index: { name: 'text', aliases: 'text' }. Invariant: Aliases lowercased and deduplicated on save.

MONGODB SCHEMA CATALOGUE

Career Intelligence & Assessment Schemas

MODEL / COLLECTION	PURPOSE & KEY FIELDS	DATA TYPES & CONSTRAINTS	INDEXES & INVARIANTS
Career careers	Occupational profile with BLS data. title, slug, domainId, summary, responsibilities[], requiredSkills[], salary, jobOutlook, tools[], traits[], status.	domainId: ObjectId → Domain, required. requiredSkills: [{ skillId: ObjectId, requiredLevel: 1..10, importance: 'critical' 'important' 'nice_to_have' }]. salary: { median, entry, top, currency, year: 2024, geography: 'US' }. status: Enum (draft, published, archived).	Unique: slug. Text Index: { title: 'text', summary: 'text', tools: 'text' }. Compound: { status: 1, domainId: 1, 'salary.median': 1 }. Invariant: Verified against 15 BLS occupational profiles.
Quiz & QuizQuestion quizzes quiz_questions	Versioned psychometric quiz & items. title, version, status, questions[]. Question: key, prompt, category, options[] (with hidden trait & domainWeights).	version: String, required, unique (e.g. v1.0). status: Enum (draft, published, archived). options[].trait: Enum (creative, analytical, people, technical, communication, empathy, organization).	Unique: version. Index: { status: 1, version: 1 }. Security: Public API strips trait and domainWeights to prevent client-side answer reverse-engineering.
QuizAttempt quiz_attempts	Learner assessment execution. userId, quizVersionId, questionSnapshots[], answers[], status, startedAt, completedAt.	userId: ObjectId → User, required. answers: [{ questionKey, optionKey, answeredAt }]. status: Enum (in_progress, completed, abandoned). questionSnapshots: Frozen copy of questions at attempt start.	Index: { userId: 1, status: 1, startedAt: -1 }. Invariant: Immutable question snapshot guarantees historical score explainability even if quiz changes.
CareerPassport career_passports	Durable competency passport. userId, sourceAttemptId, archetype, traitScores[], domainScores[], skills[], completionPercent, algorithmVersion.	userId: ObjectId → User, required. sourceAttemptId: ObjectId → QuizAttempt, unique. archetype: String (e.g. Insight Explorer). completionPercent: Number (0..100). algorithmVersion: String (passport-v1).	Unique: sourceAttemptId. Index: { userId: 1, calculatedAt: -1 }. Deterministic: Trait & domain scores normalized to 0–100 scale based on attempt answers.
RecommendationSnapshot recommendation_snapshots	Immutable ranked career matches. userId, passportId, sourceAttemptId, matches[], algorithmVersion, calculatedAt.	matches: [{ careerId, compatibilityScore, readinessScore, confidence, components[], reasons[], skillGap[] }]. algorithmVersion: String (recommendation-v1).	Unique: passportId. Index: { userId: 1, calculatedAt: -1 }. Invariant: Contains top 10 ranked careers with complete mathematical factor breakdowns.

MONGODB SCHEMA CATALOGUE

Personalization, Content, Community & Governance

MODEL / COLLECTION	PURPOSE & KEY FIELDS	DATA TYPES & CONSTRAINTS	INDEXES & LIFECYCLE RULES
Bookmark bookmarks	Polymorphic saved items & private notes. userId, itemType, itemId, itemModel, note, tags[].	itemType: Enum (career, resource, media, story). itemModel: Derived enum (Career, Resource, Multimedia, SuccessStory). note: String (max 1000 chars).	Compound Unique: { userId: 1, itemType: 1, itemId: 1 }. Index: { userId: 1, createdAt: -1 }. Invariant: Prevents duplicate bookmarks per item.
RecentlyViewed recently_viewed	Activity trail for dashboard return. userId, itemType, itemId, itemModel, viewedAt.	itemType: Enum (career, resource, media, story). viewedAt: Date (default: Date.now).	Index: { userId: 1, viewedAt: -1 }. Upsert Rule: Re-viewing an item touches viewedAt rather than creating redundant documents.
SavedFilter & Comparison saved_filters comparisons	Named career filters and comparison sets. userId, name, filters (domain, skills, salary, demand). Comparison: userId, careerIds[] (2..5 items).	filters: { domainId, skills[], minSalary, demand }. careerIds: Array of ObjectId → Career (length 2..5).	Index: { userId: 1, createdAt: -1 }. Validation: Comparison pre-save hook enforces minimum 2 and maximum 5 careers.
Resource resources	Original PDF guides and workbooks. title, slug, category, audience, file, pageCount, views, downloads, status.	file: { url, originalName, size, mimeType }. audience: Enum (student, graduate, professional, all). status: Enum (draft, published, archived).	Unique: slug. Compound: { status: 1, audience: 1 }. Counters: Atomic \$inc operations on views and downloads.
Multimedia & Rating multimedia media_ratings	Expert video media and ratings. title, slug, type, url, publisher, averageRating, ratingCount. Rating: userId, mediaId, rating (1..5).	type: Enum (video, audio, explainer). publisher: String (e.g. Google Career Certificates). rating: Number (integer 1..5).	Unique: slug. Compound Unique: { userId: 1, mediaId: 1 }. Pre-save: Media rating triggers recalculation of averageRating.
SuccessStory success_stories	Community transition stories. userId, authorName, careerId, domainId, timeline, narrative, status, isDemo.	status: Enum (pending, approved, changes_requested, rejected). narrative: { challenges, turningPoint, outcome, advice }.	Index: { status: 1, domainId: 1, isFeatured: 1 }. Moderation: Public endpoint only serves status: 'approved' records.
Feedback & Notification feedback notifications	User feedback tickets and notifications. userId, category, message, status, response. Notification: userId, type, title, body, read.	category: Enum (bug, feature, career_data, other). status: Enum (new, in_progress, resolved, closed). read: Boolean (default: false).	Index: { userId: 1, read: 1, createdAt: -1 }. Index: { status: 1, createdAt: -1 }. Lifecycle: Staff response triggers automatic Notification creation.
Settings & AuditLog settings audit_logs	System config & governance audit. key, maintenanceMode, allowRegistration. AuditLog: actorId, action, target, details, ipAddress.	key: String, unique (singleton: system_settings). actorId: ObjectId → User. action: String (e.g. user.role_change).	Unique: key. Compound: { action: 1, createdAt: -1 }. Append-Only: Audit logs are strictly immutable for compliance.

IDENTITY & ACCESS CONTROL

End-to-End Authentication & Session Lifecycle

- 1

Registration & Password Hashing: User submits name, email, password, and stage. Backend normalizes email to lowercase, verifies uniqueness, hashes password with Bcrypt (cost 12), and creates a `pending_verification` user document.
- 2

Cryptographic OTP Generation & Delivery: `auth.service.js` generates a 6-digit numeric OTP via `crypto.randomInt(100000, 999999)`, computes SHA-256 hash, stores in `VerificationToken` (+15 min TTL), and dispatches email via Resend (or console in dev).
- 3

OTP Verification & Account Activation: User enters OTP; server verifies token hash, confirms attempts < 5, marks `emailVerified: true`, updates user status to `active`, and burns the token.
- 4

Login & Secure Session Creation: On valid credential check, server generates a high-entropy 32-byte session token via `crypto.randomBytes(32).toString('hex')`, stores SHA-256 hash in `Session` (+7 day TTL), and writes raw token to an `HttpOnly` cookie.
- 5

Authenticated Request & RBAC Resolution: `requireAuth` middleware extracts session cookie, hashes value, looks up active non-revoked session, loads `User`, and `requireRole` enforces role permissions (e.g. `staff`, `admin`).
- 6

Session Termination & Multi-Session Revocation: Logout revokes the calling session token. Password reset generates a single-use SHA-256 reset token, updates password hash, and invalidates all active sessions for that user.

Authentication Security Matrix & Environment Behaviors

SECURITY CONCERN	DEVELOPMENT IMPLEMENTATION	PRODUCTION IMPLEMENTATION	SOURCE CODE EVIDENCE
Cookie Security Flags	<code>HttpOnly: true</code> , <code>SameSite: 'lax'</code> , <code>Secure: false</code>	<code>HttpOnly: true</code> , <code>SameSite: 'strict'</code> (or <code>'none'</code> if cross-domain), <code>Secure: true</code>	<code>session.service.js</code> checks <code>NODE_ENV === 'production'</code>
CORS Protection	Allows <code>http://localhost:5173</code> with <code>credentials: true</code>	Strict allowlist matching exact configured origins in <code>FRONTEND_ORIGINS</code>	<code>app.js</code> dynamic CORS origin validator
Brute-Force Rate Limiting	Relaxed window for local test automation	<code>express-rate-limit</code> : 10 attempts per 15 minutes per IP on auth routes	<code>rateLimit.middleware.js</code> (<code>authRateLimiter</code>)
Password Storage	Bcrypt salt rounds 12 (deterministic test speed)	Bcrypt salt rounds 12 (adaptive cryptographic cost)	<code>auth.service.js</code> (<code>bcrypt.hash(password, 12)</code>)

DETERMINISTIC ALGORITHMS

Scoring Mathematical Formulas & Snapshot Rules

PathSeeker employs 100% deterministic, explainable mathematics (Algorithm: `passport-v1` and `recommendation-v1`) without non-reproducible black-box heuristics.

Passport-v1 Scoring Formulas

```
TraitScore(t) = round( (Count(t) / MaxTraitCount) * 100 )
DomainScore(d) = round( (TotalWeight(d) / MaxDomainWeight) * 100 )
CompletionPercent = round( (EvidenceSignals / 7) * 100 )
```

7 Evidence Signals: Headline, Education entries, Skills list, Interests, Primary goal, Completed onboarding, Fully answered assessment.

Recommendation-v1 Formulas

```
TraitAlignment = mean( TraitScore(career.traits) )
Compatibility = round( (TraitAlign * 0.55) + (Interests * 0.25) + (SkillAffinity * 0.20) )
Readiness = round( ( Σ min(1, userSkill / reqLevel) * weight ) / Σ weight * 100 )
Confidence = round( 55 + min(40, evidenceCount * 4) )
```

Weights: Nice-to-have = 1, Important = 2, Critical = 3.

Psychometric Archetype Mapping (Based on Top Normalized Trait)

PRIMARY TRAIT	ASSIGNED ARCHETYPE	PSYCHOMETRIC DEFINITION & IDEAL CAREER CONTEXT
Creative	Imaginative Builder	Generative, visual, and design-oriented problem solving (UX Designer, Game Developer).
Analytical	Insight Explorer	Empirical data modeling, pattern recognition, and logic (Data Scientist, Financial Analyst).
People	Collaborative Leader	Interpersonal alignment, group motivation, and facilitation (Product Manager, HR Specialist).
Technical	Practical Innovator	Applied systems engineering, tool execution, and architecture (DevOps, Cloud Engineer).
Communication	Clear Connector	Structured narrative, technical writing, and stakeholder alignment (Technical Writer, PR).
Empathy	Thoughtful Builder	User-centered advocacy, accessibility, and human care (Healthcare Informatics, Counselor).
Organization	Structured Strategist	Operational discipline, roadmapping, and milestone delivery (Operations Lead, IT Director).

Skill Gap Analysis & Simulator Safety

Skill Gap Mathematical Delta

For each career prerequisite: `Difference = max(0, requiredLevel - currentLevel)`.

```
ReadinessImpact = clamp( (Difference / requiredLevel) * weight * 20 )
```

Gaps are sorted by highest `ReadinessImpact` first to prioritize immediate learning needs.

Stateless Simulator Guarantee

The interactive skill simulator accepts temporary level overrides, executes `scoreCareer()` in memory, and returns `{ before, after, persisted: false }`.

Safety Invariant: It never mutates the underlying `UserProfile`, `CareerPassport`, or `RecommendationSnapshot`.

SUBSYSTEM ORCHESTRATION

Content Workflows & Admin Control Plane

Original PDF Resource Workflow

- 6 original downloadable career workbooks (51 pages total).
- In-browser canvas preview with direct attachment download.
- Tracks audience tags, page count, and real-time view/download counters.
- Curated relationships directly link workbooks to target careers.

Expert Multimedia Subsystem

- 6 official Google Career Certificates expert video records.
- Embedded via privacy-enhanced YouTube player with direct source links.
- Includes learning objectives, key notes, and captions guidance.
- Authenticated 1–5 star rating system with atomic average recalculation.

Success Story Moderation State Machine



Administrative Control Plane Workspaces

ADMIN MODULE	ACCESS LEVEL	CAPABILITIES & GOVERNANCE RULES
User Governance	admin / super_admin	Search, filter by status/role, promote/demote user roles, suspend/activate accounts. Generates immutable AuditLog .
Career Bank Management	staff (all)	Create, edit, and delete occupational profiles; manage required skills and salaries; transition publication states.
Quiz Builder	staff (all)	Create and reorder questions, preview scoring weights, publish immutable new quiz versions (e.g. v1.1), archive old versions.
Resource & Media CRUD	staff (all)	Upload and update PDF workbooks; attach metadata, video URLs, and learning objectives; toggle publication.
Story & Feedback Triage	staff (all)	Approve/reject stories with moderation notes; assign feedback tickets, write internal notes, send learner responses.
System Settings & Logs	admin / super_admin	Toggle site-wide maintenance mode and registration; inspect paginated immutable security audit logs.

CONVERSATIONAL UX & INCLUSION

Navi Guidance Agent & Inclusive Design

Deterministic Navi Architecture

Navi is PathSeeker's character-driven guidance assistant. Unlike hallucination-prone external chatbots, the default backend assistant is **100% deterministic**, classifying learner queries into grounded intent categories with safe route navigation.

Optional External Voice Mode

Frontend integrates an optional Vapi WebSocket client. When voice mode is toggled, it streams conversational audio. If voice credentials or microphone permissions are unavailable, Navi falls back cleanly to text mode without blocking UX.

Navi Intent Classification & Action Contract

INTENT CLASS	SAMPLE LEARNER QUERY	EXECUTED SYSTEM ACTION	AUTH RULE
Navigation	"Take me to the career quiz"	Returns action { type: 'navigate', path: '/app/quiz' }.	Public/Learner
Career Search	"Find cloud engineering jobs"	Executes grounded search query against <code>Career</code> collection.	Public
Product Help	"How does the simulator work?"	Returns grounded explanation of non-persistent skill simulation.	Public
Profile Mutation	"Add Python to my skills"	Extracts skill, validates existence, appends to <code>UserProfile</code> .	<code>requireAuth</code>
Fallback	"What is the capital of France?"	Gracefully redirects learner to supported career topics.	Public

Accessibility (WCAG 2.1 AA) Implementation

Font Scaling

CSS variable `--app-font-scale` adjusts UI text smoothly from 85% to 130% without layout breakage.

Theme Sync

Supports explicit Dark, Light, and System OS preference modes with persistent `localStorage` sync.

Reduced Motion

Honors `prefers-reduced-motion`, suppressing transitions and animations across cards and dialogs.

Keyboard Safe

Visible focus rings, logical tab indices, skip-to-content links, and Escape-key dismiss on modal dialogs.

Safety Boundary Disclaimer: Navi is designed solely for educational navigation and career exploration. It is not an automated employment recruiter, licensed psychological counselor, or mental-health diagnostic tool.

THREAT MODEL & ERROR CONTRACTS

Threat-Informed Security Matrix & Error Standards

THREAT CATEGORY	ATTACK SURFACE	PREVENTIVE CONTROL IN SOURCE	DETECTION & RECOVERY
Credential Compromise	/api/auth/login	Bcrypt hashing (cost 12), rate limiting, generic invalid credential error message.	Revoke all sessions on password reset; audit login timestamps.
Session Hijacking	HttpOnly Cookies	32-byte high-entropy token, SHA-256 hash stored at rest, SameSite flag, 7-day TTL.	Immediate server-side token revocation on logout; TTL index cleanup.
Privilege Escalation	/api/admin/*	requireRole middleware checks authenticated user role from fresh database query.	403 Forbidden event logged with actor ID and client IP.
Brute-Force Abuse	OTP & Auth routes	authRateLimiter (10 req/15m) + 5-attempt ceiling on OTP verification tokens.	429 Too Many Requests response; temporary IP cooldown window.
NoSQL Injection	REST parameters	Mongoose schema strict typing, Zod schema validation, object ID casting checks.	Mongoose CastError caught by global error middleware (returns 400).
Cross-Origin Exploitation	Browser API calls	CORS middleware enforces strict origin allowlist from FRONTEND_ORIGINS env.	Non-allowlisted origins rejected during preflight OPTIONS check.
Malicious File Upload	Avatar / Resume upload	uploadSingleFile enforces 5MB limit, whitelist MIME types (PDF, JPEG, PNG).	Multer rejects invalid files before disk persistence; cleans temp files.
Score Manipulation	Quiz completion	Server owns question weights and scoring logic; client only submits option keys.	Immutable quiz snapshot prevents retroactive manipulation.

Standardized REST Error Envelope

```
// Standard Safe Error Payload
{
  "error": {
    "message": "Invalid verification code or code expired.",
    "code": "INVALID_OTP",
    "status": 400,
    "details": [],
    "requestId": "3628dd8e-ba17-4753-af5f-bf850e319326"
  }
}
```

HTTP Status Semantics

- 400 Bad Request — Malformed input, Zod validation failure, invalid OTP.
- 401 Unauthorized — Missing or expired session cookie.
- 403 Forbidden — Insufficient role permissions for target resource.
- 404 Not Found — Requested career slug or resource ID does not exist.
- 409 Conflict — Duplicate email registration or existing bookmark.
- 429 Too Many Requests — Exceeded rate limit thresholds.
- 500 / 503 Server Error — Unhandled exception or database offline.

QUALITY VERIFICATION

Formal Test Case Specification (Auth, Quiz, Intelligence)

TEST ID	SRS ID	MODULE	TEST DATA / PRECONDITIONS	EXECUTION STEPS & EXPECTED RESULT	STATUS & EVIDENCE
TC-AUTH-01	FR-01	Auth Service	<pre>{ name, email: "new@user.com", password: "Password123!", stage: "student" }</pre>	Submit registration. Assert user is created with <code>pending_verification</code> , password hashed, 6-digit OTP created.	VERIFIED <code>models.test.js</code>
TC-AUTH-02	FR-01	Auth Service	Duplicate email registration.	Submit existing email. Assert API returns 409 Conflict with code <code>EMAIL_EXISTS</code> .	VERIFIED <code>models.test.js</code>
TC-AUTH-03	FR-01	Auth Service	Malformed OTP (empty or invalid string).	POST <code>/api/auth/verify-email</code> . Assert 400 Bad Request returned before database access.	VERIFIED <code>app.smoke.test.js</code>
TC-AUTH-04	FR-01	Auth Service	Valid login credentials.	POST <code>/api/auth/login</code> . Assert 200 OK, returns user DTO, sets <code>HttpOnly</code> session cookie.	VERIFIED <code>app.smoke.test.js</code>
TC-AUTH-05	FR-01	Session Service	Expired session token.	Query protected route with expired token. Assert 401 Unauthorized and cookie clearance.	VERIFIED <code>models.test.js</code>
TC-QUIZ-01	FR-04	Quiz Service	Published quiz (Quiz v1).	GET <code>/api/quiz-questions</code> . Assert 200 OK, returns questions with hidden weights stripped.	VERIFIED <code>seeds.test.js</code>
TC-QUIZ-02	FR-04	Quiz Attempt	Create attempt & answer questions.	PATCH <code>/api/quiz-attempts/:id/answers</code> . Assert answers append and question snapshot is frozen.	VERIFIED <code>models.milestone2.test.js</code>
TC-QUIZ-03	FR-04	Quiz Attempt	Submit completed attempt.	POST <code>/api/quiz-attempts/:id/complete</code> . Assert <code>completedAt</code> stamped, triggers Passport generation.	VERIFIED <code>models.milestone2.test.js</code>
TC-INTEL-01	FR-11	Scoring Engine	Analytical answers + Python skill.	Calculate passport. Assert <code>archetype: "Insight Explorer"</code> and <code>completionPercent: 100</code> .	VERIFIED <code>career-career-intelligence.test.js</code>
TC-INTEL-02	FR-11	Scoring Engine	Deterministic repeatability.	Execute calculation 100 times on identical data. Assert scores and archetype remain identical.	VERIFIED <code>career-career-intelligence.test.js</code>
TC-INTEL-03	FR-11	Simulator	Skill override adjustments.	POST <code>/api/users/me/careers/:slug/simulate</code> . Assert readiness increases and <code>persisted: false</code> .	VERIFIED <code>career-career-intelligence.test.js</code>

QUALITY VERIFICATION

Formal Test Case Specification (Personalization, Security, UI)

TEST ID	SRS ID	MODULE	TEST DATA / PRECONDITIONS	EXECUTION STEPS & EXPECTED RESULT	STATUS & EVIDENCE
TC-PERS-01	FR-09	Bookmark Service	Career ID + private note.	POST <code>/api/users/me/bookmarks</code> . Assert bookmark created with derived <code>itemModel: 'Career'</code> .	VERIFIED <code>models.milestone2.test.js</code>
TC-PERS-02	FR-09	Comparison	1 single career ID.	POST <code>/api/users/me/comparisons</code> with 1 career. Assert Mongoose validation rejects (<2 items).	VERIFIED <code>models.milestone2.test.js</code>
TC-PERS-03	FR-09	Comparison	3 valid career IDs.	POST <code>/api/users/me/comparisons</code> . Assert comparison saved successfully.	VERIFIED <code>models.milestone2.test.js</code>
TC-CONT-01	FR-07	Resource Service	Original PDF guides.	Verify seeded resources. Assert all 6 PDF files exist on disk and total 51 valid renderable pages.	VERIFIED <code>seeds.test.js</code>
TC-CONT-02	FR-05	Media Service	Expert YouTube videos.	Verify multimedia seeds. Assert 6 Google Career Certificates records with valid video IDs.	VERIFIED <code>seeds.test.js</code>
TC-SEC-01	FR-10	RBAC Middleware	Unauthenticated request to <code>/api/admin/*</code> .	GET <code>/api/admin/careers</code> without cookie. Assert 401 Unauthorized returned.	VERIFIED <code>ported-features.smoke.test.js</code>
TC-SEC-02	FR-10	RBAC Middleware	Learner role requesting admin route.	GET <code>/api/admin/users</code> with role <code>user</code> . Assert 403 Forbidden returned.	VERIFIED <code>ported-features.smoke.test.js</code>
TC-SEC-03	NFR-05	Health Probe	Database offline simulation.	GET <code>/api/health/db</code> while DB disconnected. Assert 503 Service Unavailable returned.	VERIFIED <code>app.smoke.test.js</code>
TC-UI-01	FR-01	Router Component	Direct URL navigation to <code>/login</code> .	Render router with <code>/login</code> path. Assert Login page renders with accessible form elements.	VERIFIED <code>RouterApp.test.jsx</code>
TC-UI-02	FR-12	Navi Assistant	Typed guidance query in widget.	Enter "How to explore careers". Assert Navi returns matching intent and navigation button.	VERIFIED <code>NaviAssistant.test.jsx</code>
TC-UI-03	NFR-02	Accessibility	Font scale slider mutation.	Change font scale to 120%. Assert root CSS variable <code>--app-font-scale</code> updates to <code>1.2</code> .	VERIFIED Manual & Component Check

Summary of Automated Quality Evidence

36 / 36 Backend Automated Tests (Node Test Runner)	11 / 11 Frontend Unit Tests (Vitest & Testing Library)	0 ESLint Syntax & Rule Violations	51 / 51 Rendered PDF Workbook Pages
---	--	---	---

EVALUATOR RUNBOOK

Installation Guide, Environment & Deployment

Local Reproducible Setup Sequence

- 1

Prerequisites: Ensure Node.js 20.x LTS or higher, npm 10.x+, and MongoDB 7.0+ (local daemon or Atlas URI) are available.
- 2

Backend Configuration & Seeding: Navigate to `Project/backend1`. Copy `.env.example` to `.env`. Run `npm install`, then execute `npm run seed` to populate the database with canonical careers, quizzes, and demo personas.
- 3

Start Backend Server: In `Project/backend1`, run `npm start`. Verify API readiness at `http://localhost:4000/api/health` (status: `ok`) and `/api/health/db` (status: `ready`).
- 4

Frontend Configuration & Startup: Navigate to `Project/frontend`. Copy `.env.example` to `.env` (default: `VITE_API_URL=/api`). Run `npm install`, then `npm run dev`. Open `http://localhost:5173` in your browser.
- 5

Automated Verification: Run `npm test` in `Project/backend1` (36 tests pass) and `npm test -- --run` in `Project/frontend` (11 tests pass).

Environment Configuration Matrix

VARIABLE NAME	TARGET	CLASSIFICATION	PURPOSE & DESCRIPTION
MONGODB_URI	Backend	Required	MongoDB connection string (e.g. <code>mongodb://localhost:27017</code> or Atlas URI).
MONGODB_DB_NAME	Backend	Required	Target database name (default: <code>pathseeker</code>).
PORT	Backend	Optional	Express HTTP listener port (default: <code>4000</code>).
FRONTEND_ORIGINS	Backend	Production	Comma-separated CORS allowlist (e.g. <code>https://pathseeker.vercel.app</code>).
SESSION_SECRET	Backend	Production	High-entropy secret key for cryptographic cookie signing.
RESEND_API_KEY	Backend	External / Optional	Resend API key for live transactional emails (falls back to console in dev).
VITE_API_URL	Frontend	Required	API base URL (default: <code>/api</code> for Vite proxy, or full URL in production).
VITE_VAPI_PUBLIC_KEY	Frontend	External / Optional	Vapi client key for optional voice agent functionality.

Demonstration Personas for Evaluation

PERSONA	NAME	EMAIL	PASSWORD	ROLE / STAGE	LOCATION & PROFILE SUMMARY
Super Admin	Sarah Malik	admin@pathseeker.local	PathSeeker-Demo-2026!	super_admin	Master administrator account with full governance, catalog curation, and audit log inspection.
Student	Alex Morgan	demo.student@pathseeker.local	PathSeeker-Demo-2026!	user (student)	Undergraduate Psychology student (Karachi) exploring people-centered technology careers.
Graduate	Zara Noor	demo.graduate@pathseeker.local	PathSeeker-Demo-2026!	user (graduate)	CS graduate (Lahore) building practical skills (React/JS) and closing career readiness gaps.
Professional	Daniel Kim	demo.professional@pathseeker.local	PathSeeker-Demo-2026!	user (professional)	Finance professional (Islamabad) transitioning to Data Analysis and evaluating remote roles.

AUTHORITATIVE IMPLEMENTATION AUDIT

Complete SRS Traceability Matrix

SRS ID	REQUIREMENT TITLE	FRONTEND MODULE	BACKEND MODULE	DATA MODEL	API ENDPOINT	VERIFIED STATUS
FR-01	Auth & User Mgmt	AuthContext , LoginPage	auth.service.js	User , Session	/api/auth/*	VERIFIED
FR-02	Personalized Dashboard	DashboardPage.jsx	dashboard.service.js	UserProfile	/api/users/me/dashboard	VERIFIED
FR-03	Career Bank & Search	CareersPage.jsx	catalog.service.js	Career , Domain	/api/careers/*	VERIFIED
FR-04	Psychometric Quiz	QuizPage.jsx	quiz.service.js	QuizAttempt	/api/quiz-attempts/*	VERIFIED
FR-05	Multimedia Center	MediaPage.jsx	content.service.js	Multimedia	/api/media/*	VERIFIED
FR-06	Success Stories Hub	SuccessStoriesPage.jsx	admin.service.js	SuccessStory	/api/stories/*	VERIFIED
FR-07	Resource Library (PDF)	ResourcesPage.jsx	content.service.js	Resource	/api/resources/*	VERIFIED
FR-08	Feedback & Alerts	FeedbackPage.jsx	notification.service.js	Feedback	/api/feedback/*	VERIFIED
FR-09	Bookmarks & Compare	SavedPage.jsx	personalization.service.js	Bookmark	/api/users/me/bookmarks	VERIFIED
FR-10	Admin Control Panel	/admin/*	admin.service.js	AuditLog , Settings	/api/admin/*	VERIFIED
FR-11	Career Intelligence	PassportPage.jsx	careerIntelligence.service.js	CareerPassport	/api/users/me/passport	VERIFIED
FR-12	Accessibility & Navi	AccessibilityContext	assistant.service.js	UserProfile	/api/assistant/respond	VERIFIED
NFR-01	Safety & Content	VideoEmbed.jsx	content.service.js	Resource	Asset endpoints	VERIFIED
NFR-02	WCAG Accessibility	AccessibilityProvider	Theme / CSS layers	UserProfile	Client preferences	VERIFIED
NFR-04	Security & Privacy	apiClient.js	auth.middleware.js	Session	All routes	VERIFIED
NFR-05	High Availability	Error boundaries	server.js	Database	/api/health/*	VERIFIED

Traceability Audit Result: Every functional requirement (FR-01 through FR-12) and quality constraint has been mapped directly to production modules, routes, schemas, and verified test cases.

FORMAL PROJECT SIGN-OFF

Limitations, Authority & Approval Record

Requirements Exception & Limitations Register

AREA / REQUIREMENT	EXPECTED SCOPE	CURRENT IMPLEMENTATION STATE	EVALUATOR IMPACT & NEXT STEP
Assessment Formats	Multiple question formats.	8 multiple-choice questions seeded (Quiz v1); schema supports Likert/ranking.	Core flow verified; add slider variants in subsequent release.
Transactional Email	Live email delivery.	Console email provider in dev; live Resend API configured for production.	Zero evaluation blocker; OTPs logged directly to server console.
Voice Guidance (Navi)	Bidirectional audio.	Deterministic text Navi built-in; optional Vapi voice provider integrated.	Text guidance 100% operational; voice toggles with active provider key.
Occupational Geography	Global pay datasets.	U.S. Bureau of Labor Statistics (BLS 2024–2034) reference benchmark.	Authoritative data baseline; localized datasets recommended for future scope.

Implementation Completeness Statement

Authoritative Engineering Sign-Off: PathSeeker has been fully designed, implemented, and benchmarked against the Aptech TechWiz SRS. All 12 Functional Requirements and Quality Attributes are backed by concrete source code, deterministic algorithms, 27 Mongoose schemas, 36 passed backend tests, 11 passed frontend tests, 0 lint errors, and 51 renderable PDF pages. The platform is ready for evaluator assessment and deployment.

LEAD SOFTWARE ENGINEER

Mirza Abdullah Baig

StackVerse Project Lead

Student ID: 1600671

TECHNICAL REVIEW

Baseline Verified

36/36 Backend Passed

11/11 Frontend Passed

COMPETITION BASELINE

Aptech TechWiz

Status: Ready for Evaluation

Date: 27 August 2026